

Documentación Técnica

ARGENTUM ONLINE

Taller de Programación — Cátedra Veiga — 1er cuatrimestre 2026

Aylén Bartomeo	— 111738
Franco Bustos	— 110759
Alexander Coronado	— 111256
Bruno Pezman	— 110457

Índice

1. Introducción	2
2. Organizacion del directorio	2
3. Arquitectura del Sistema	2
3.1. Servidor	2
3.2. Cliente	2
3.3. Editor	3
3.4. Diagramas	3
4. Tests	15
5. Valgrind	16

1. Introducción

Este documento detalla la arquitectura del sistema para facilitar la continuidad del desarrollo.

2. Organización del directorio

- **client/**: Cliente gráfico (SDL2 + launcher Qt5).
- **server/**: Servidor de juego.
- **editor/**: Editor de mapas.
- **common/**: Código compartido (protocolo, sockets, DTOs, etc.).
- **tests/**: Tests unitarios e integrales (GoogleTest).
- **config/**: Archivos de configuración del juego en formato .toml.
- **documents/**: Enunciado y manuales del juego.
- **maps/**: Archivos de mapas en formato JSON (incluyendo el mapa canónico: maps/defaultMap.json).
- **resources/**: Assets gráficos y recursos multimediales utilizados por el cliente y el editor.

3. Arquitectura del Sistema

3.1. Servidor

El servidor es responsable de:

- Controlar y validar de manera autoritativa la lógica del juego.
- Comunicar el estado actualizado del mundo a todos los jugadores conectados.
- Administrar la sincronización, la persistencia de datos y la coexistencia de los jugadores.

3.2. Cliente

El cliente es el encargado de:

- Representar visualmente el estado del juego y el entorno para el jugador.
- Enviar las actualizaciones y acciones del jugador hacia el servidor.
- Gestionar y utilizar las texturas, fuentes y sonidos requeridos.

3.3. Editor

El editor es el encargado de:

- Permitir la creación y edición de mapas mediante una grilla lógica de baldosas (*tiles*).
- Configurar propiedades lógicas sobre el terreno tales como zonas de colisión, pasajes y puntos de aparición (*spawn*).
- Gestionar un sistema de capas visuales para estructurar correctamente la superposición de elementos en el mundo.

3.4. Diagramas

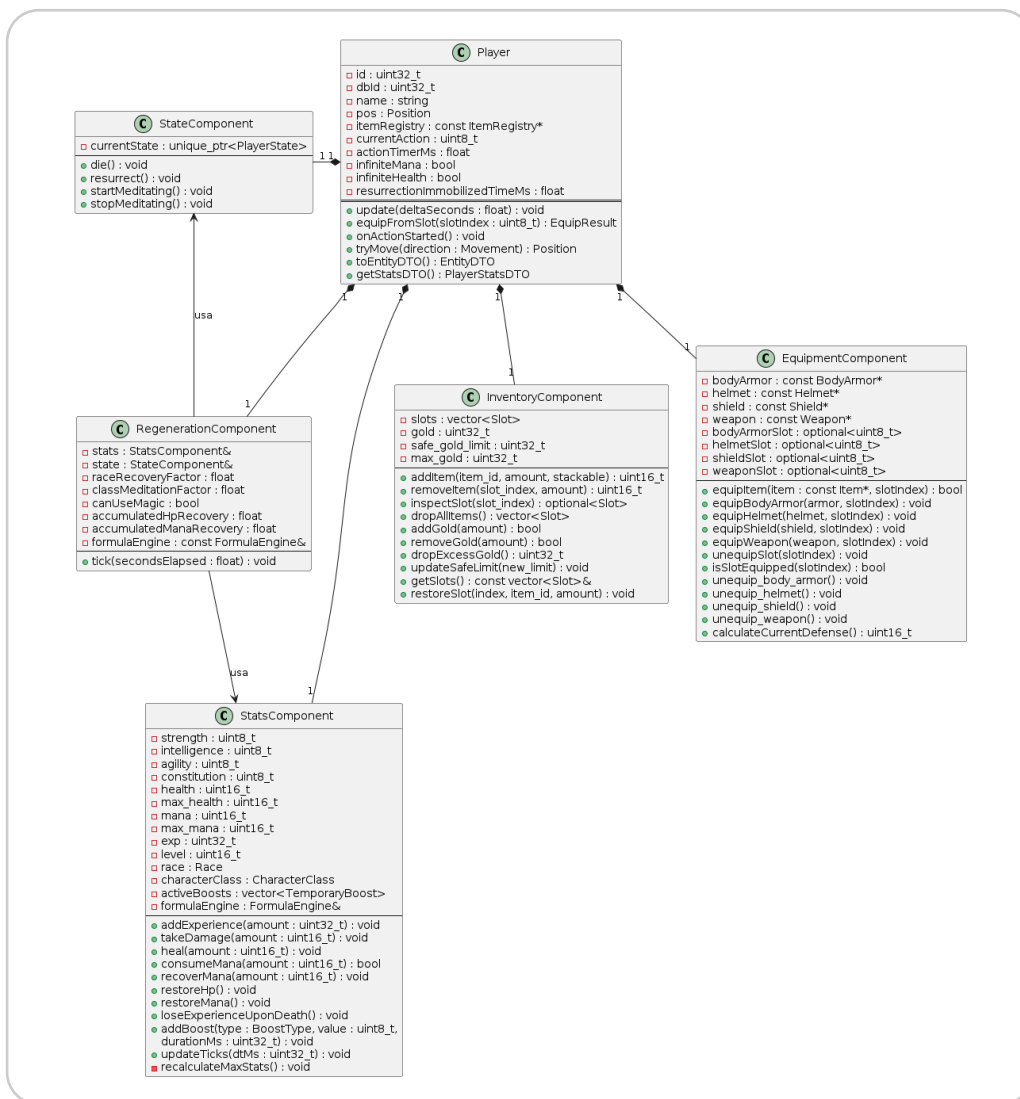


Figura 1: Diagrama de la clase *Player* y sus dependencias de componentes.

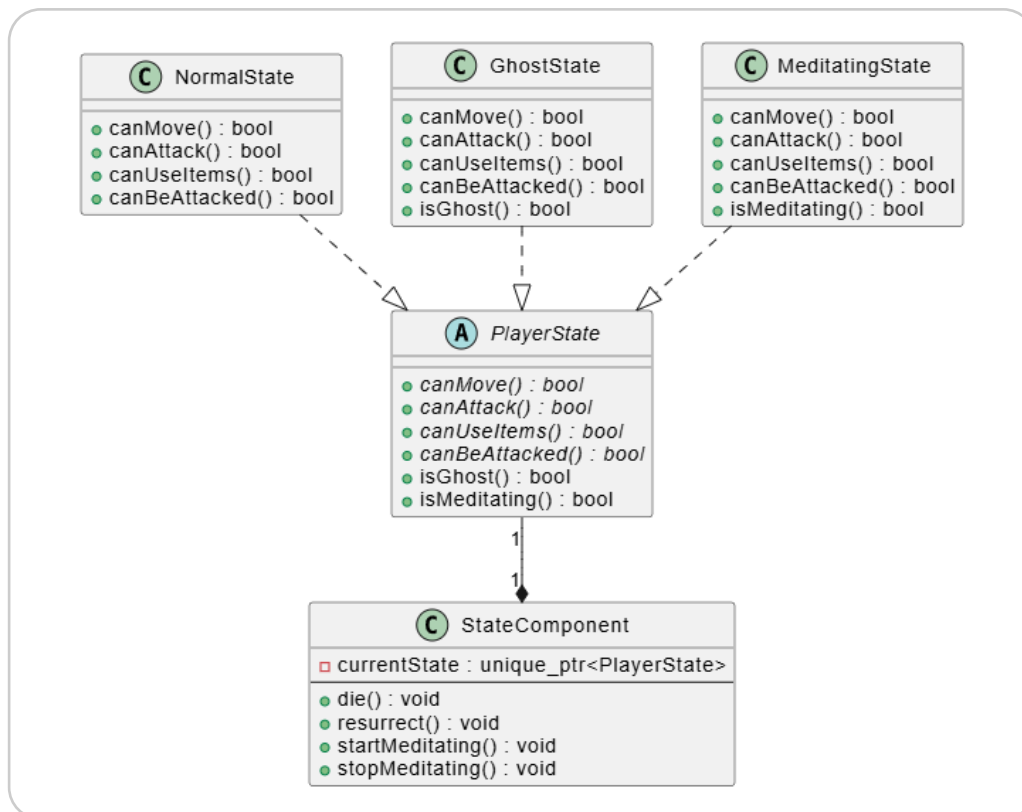


Figura 2: *Implementacion del patron State.*

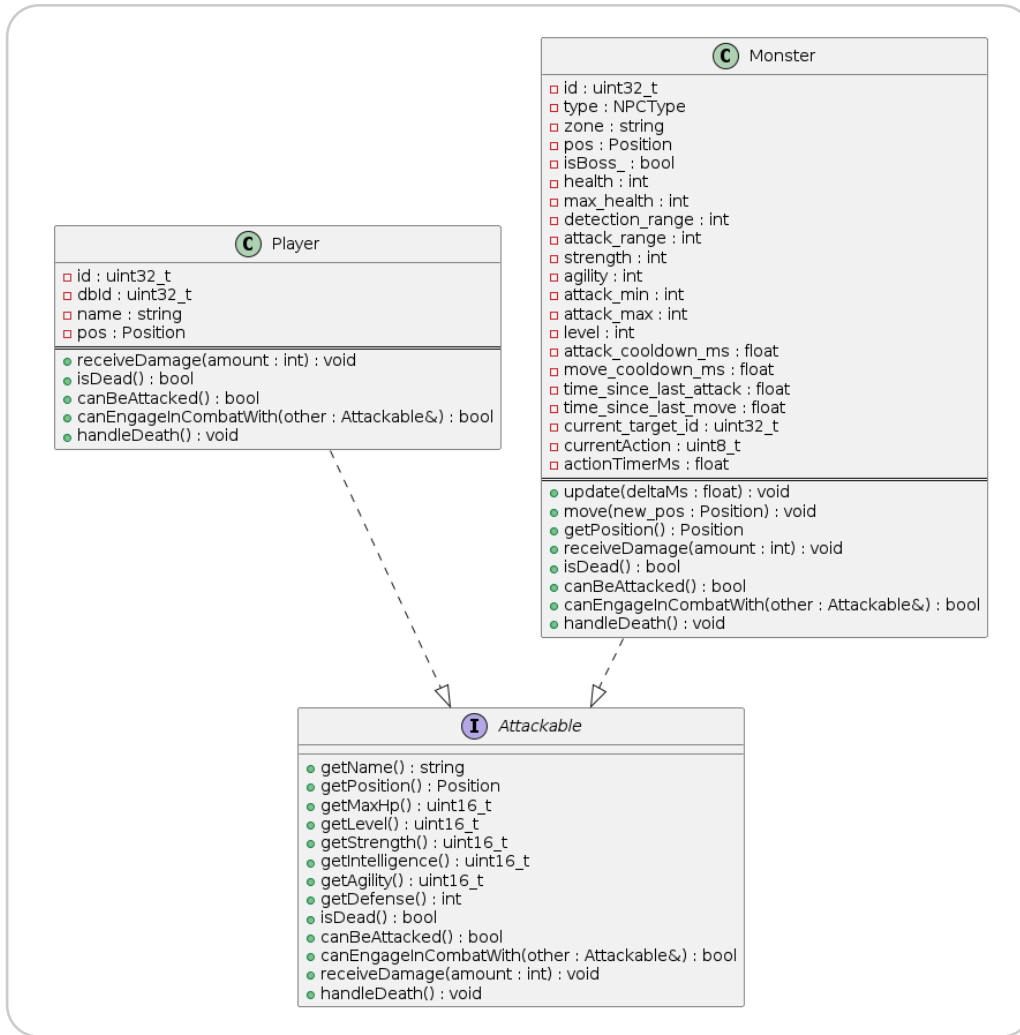


Figura 3: Contrato en comun entre Monster y Player

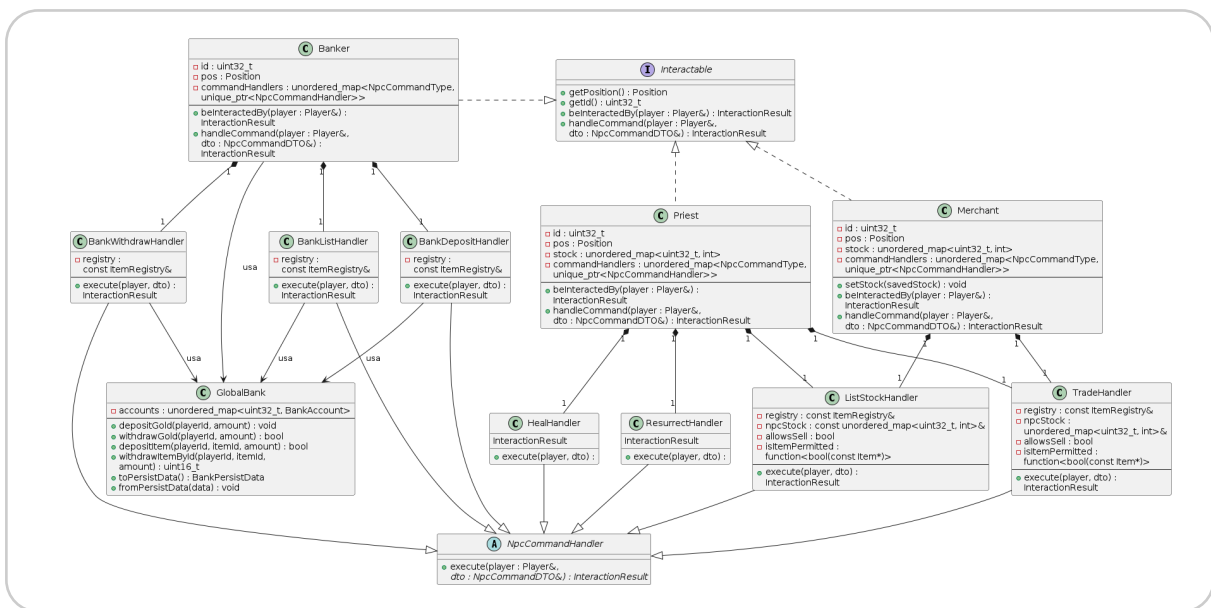


Figura 4: Los NPCs implementan la clase Interactable y el patrón Command

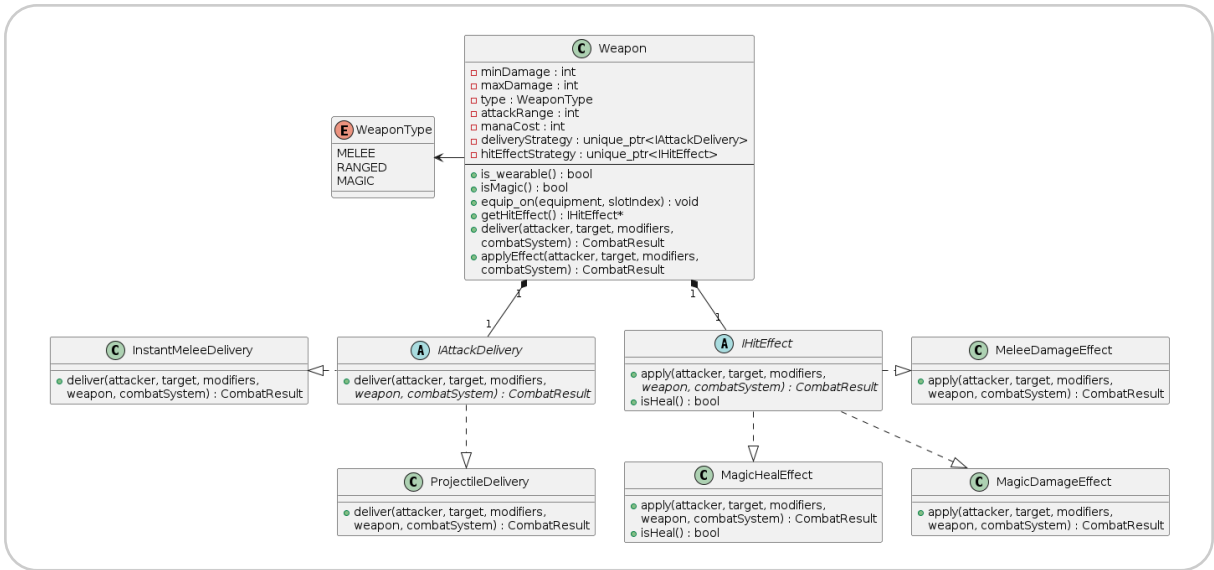


Figura 5: Implementación de doble Strategy en Weapons

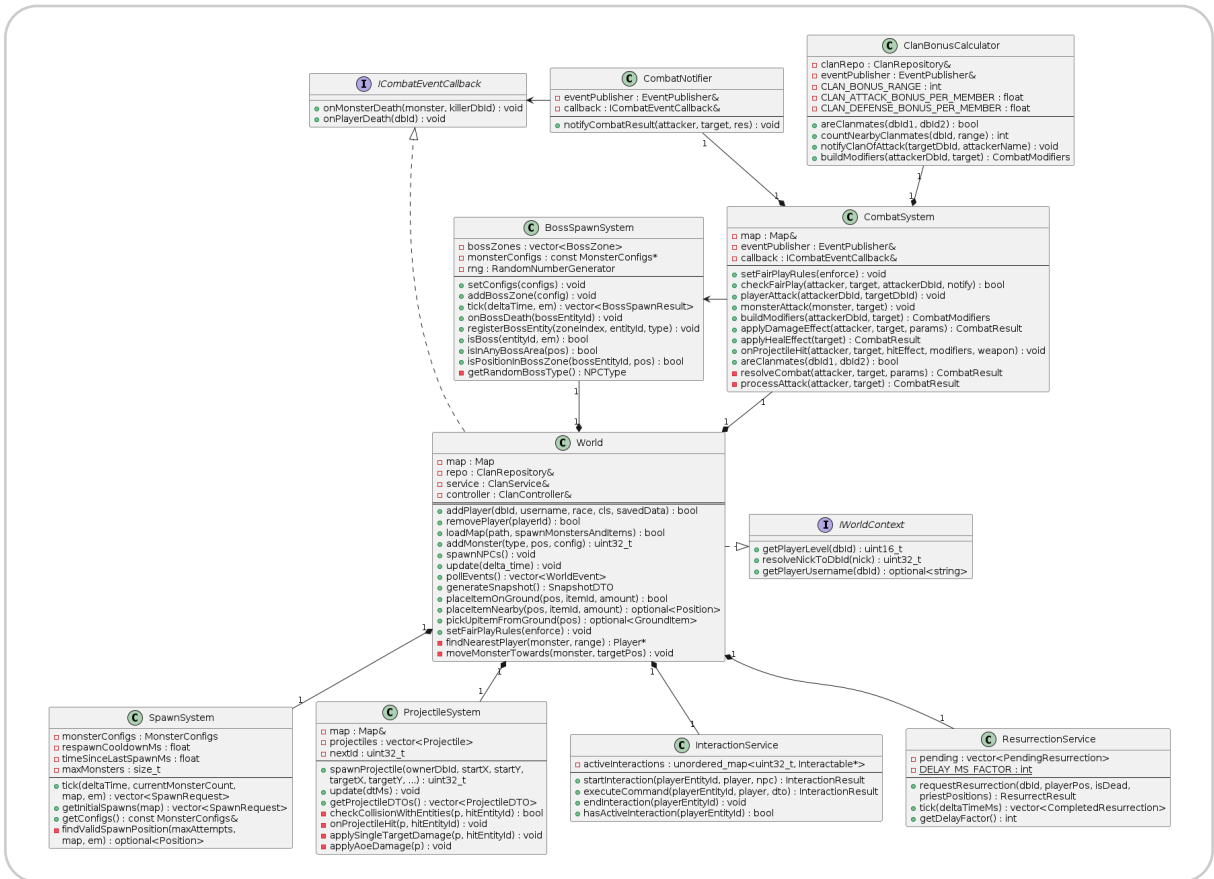


Figura 6: World como fachada, delegando la implementación de funcionalidades en sus componentes.

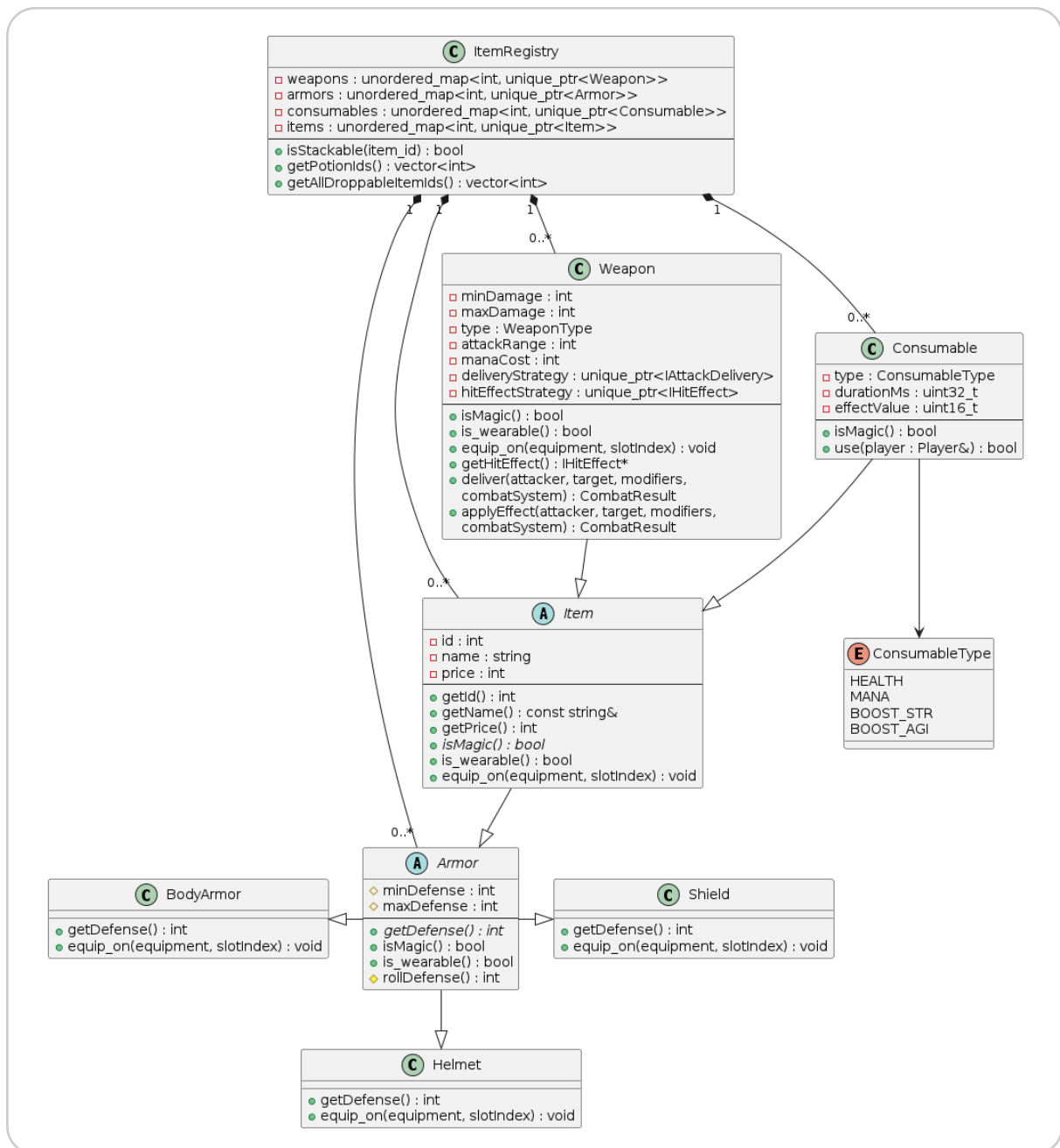


Figura 7: Esquema de la estructura de items

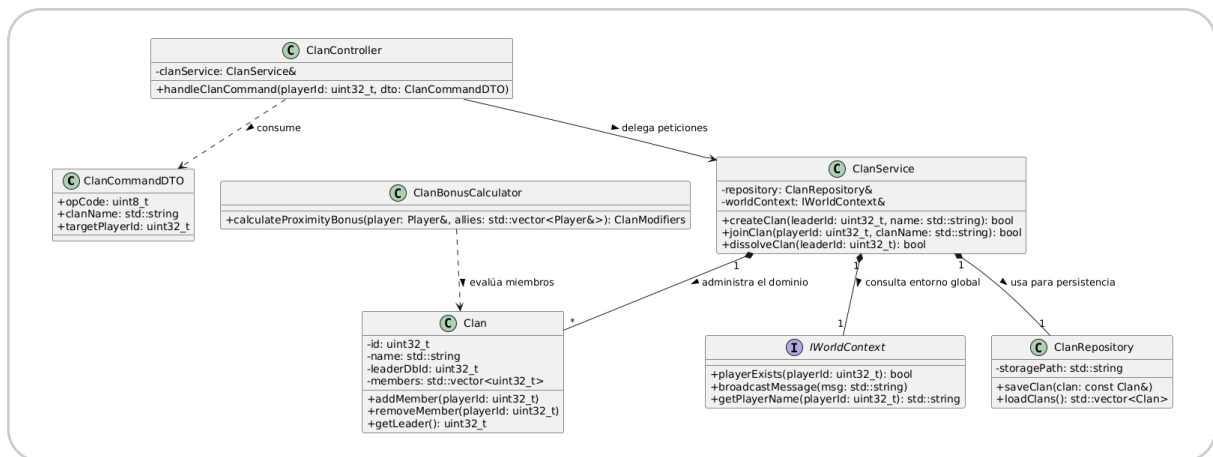


Figura 8: Estructura y delegación del sistema de clanes. Muestra la interacción entre el controlador, el manejo del dominio mediante *ClanService*, y la persistencia a través de *ClanRepository*.

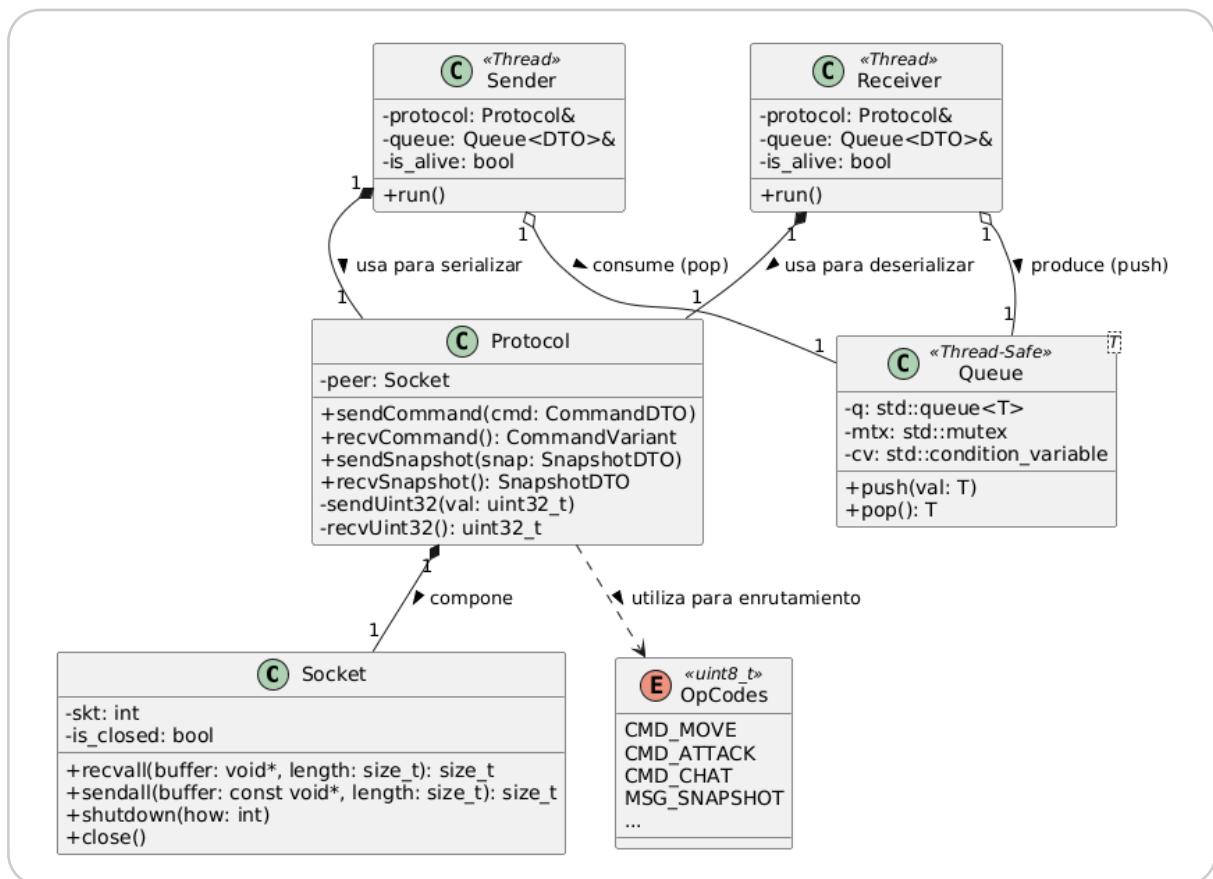


Figura 9: Arquitectura del sistema de red y protocolo. Ilustra la concurrencia mediante los hilos *Sender* y *Receiver*, el manejo de mensajes a través de una cola thread-safe (*Queue*), y la serialización/deserialización encapsulada en la clase *Protocol* utilizando *Sockets* y *OpCodes*.

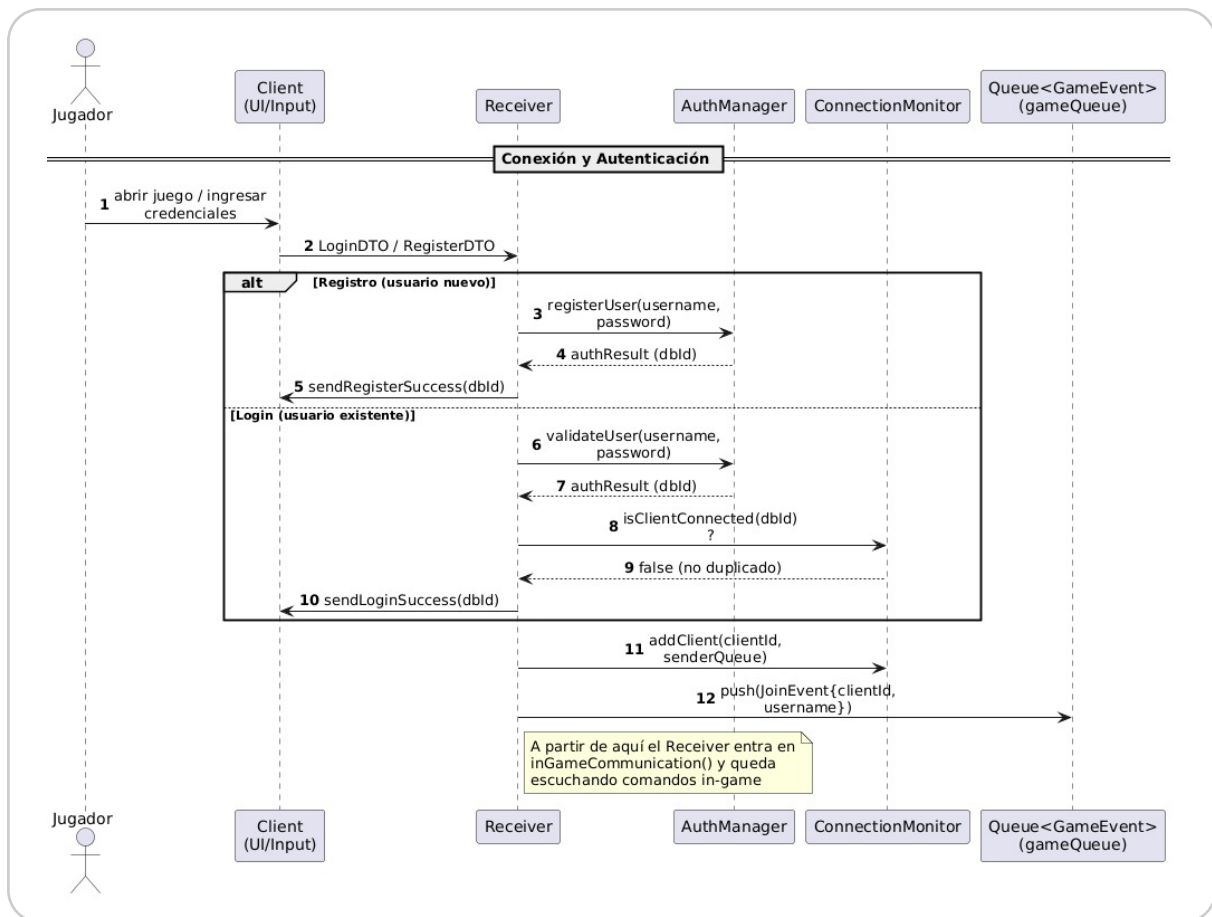


Figura 10: Inicio de sesión

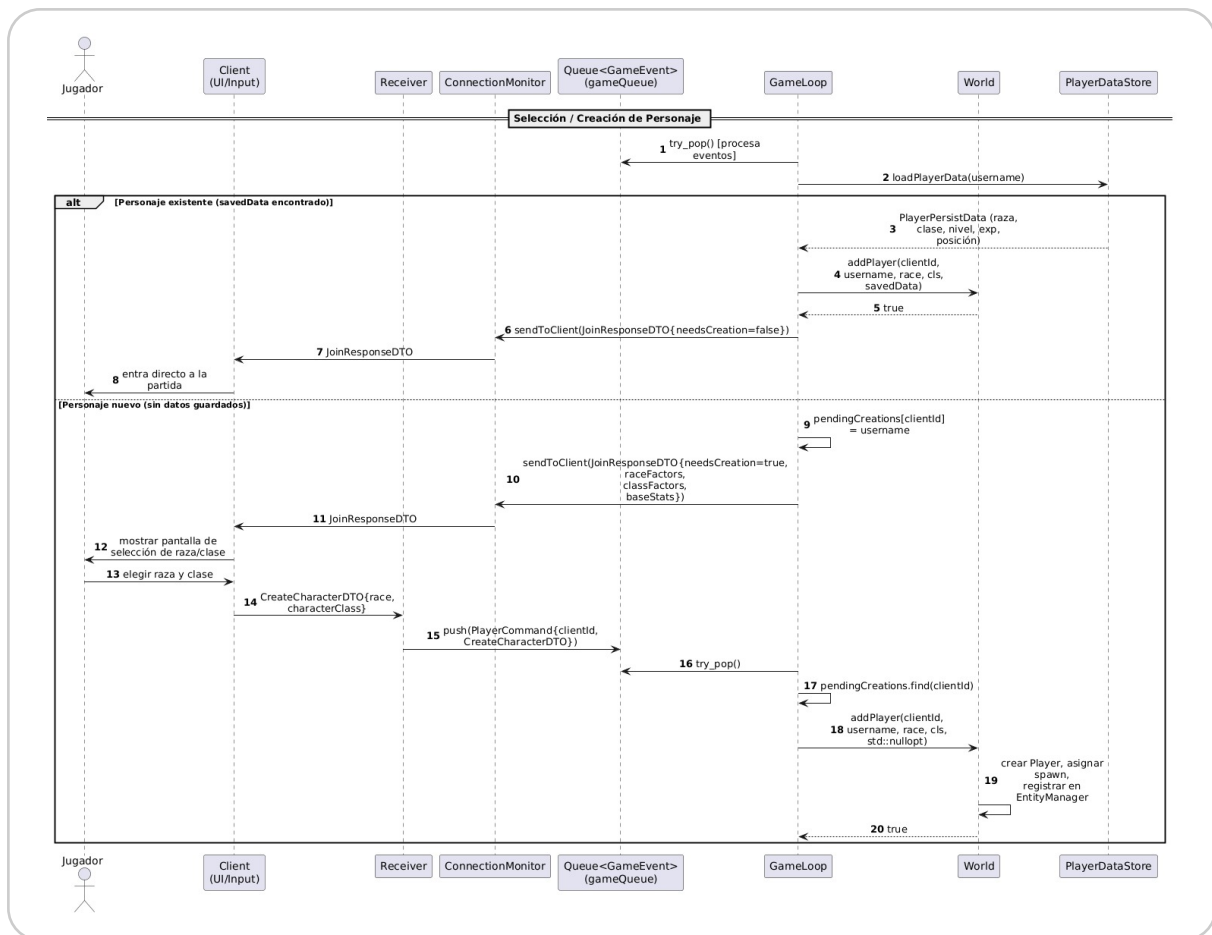


Figura 11: Flujo de seleccion de raza y clase del Player.

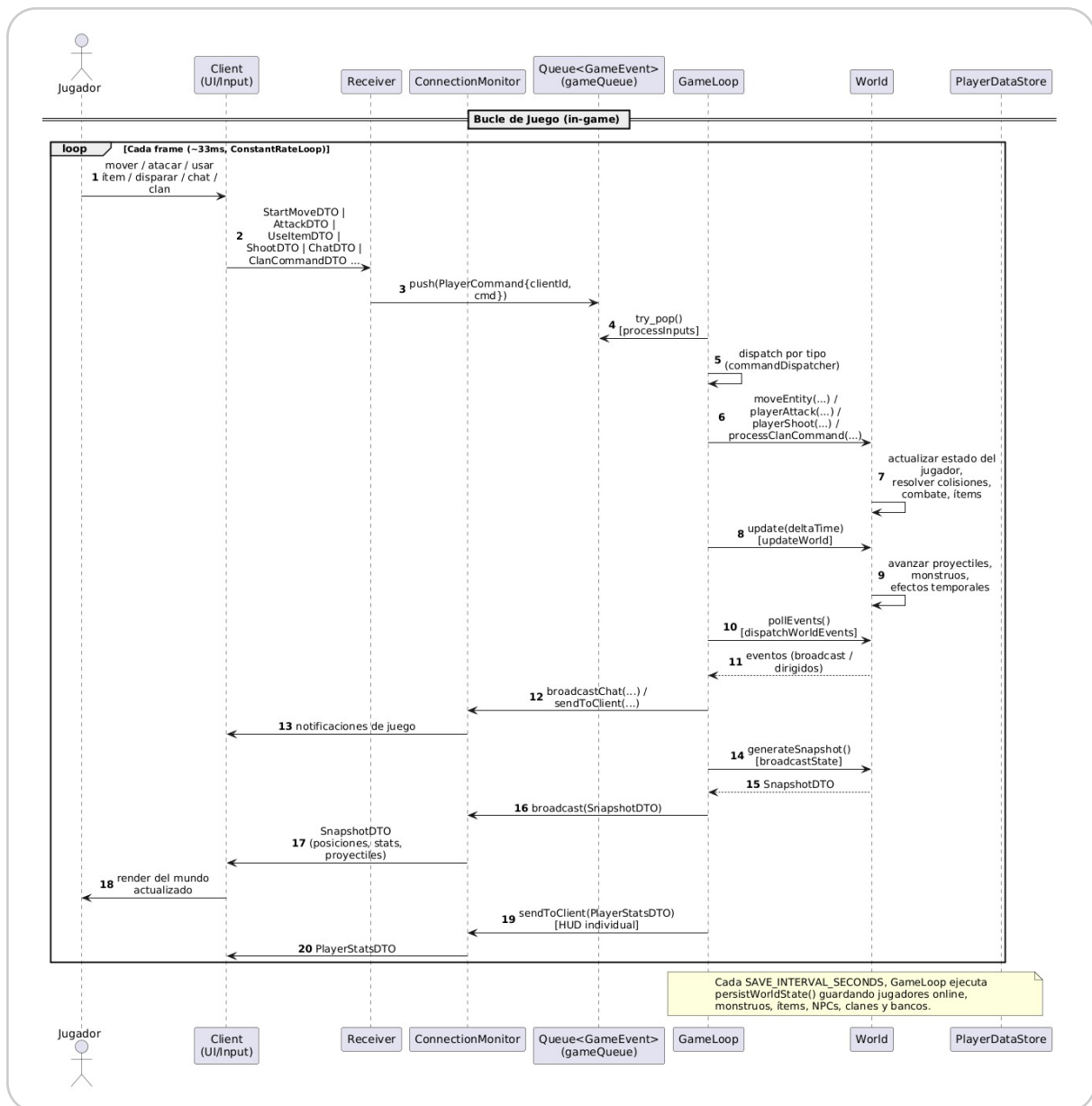


Figura 12: *Flujo del juego.* Se observa como interactúan los objetos ante el input de los comandos del usuario.

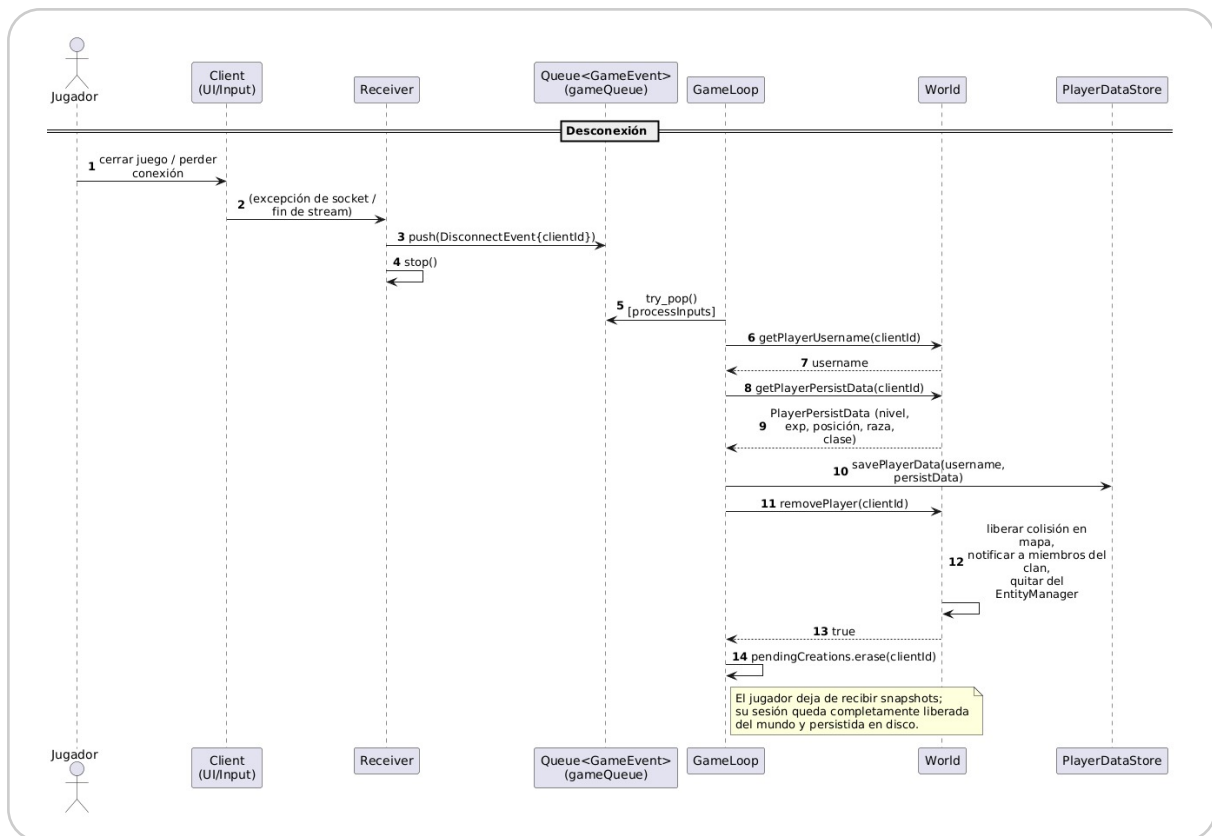


Figura 13: *Flujo del fin de la sesión.*

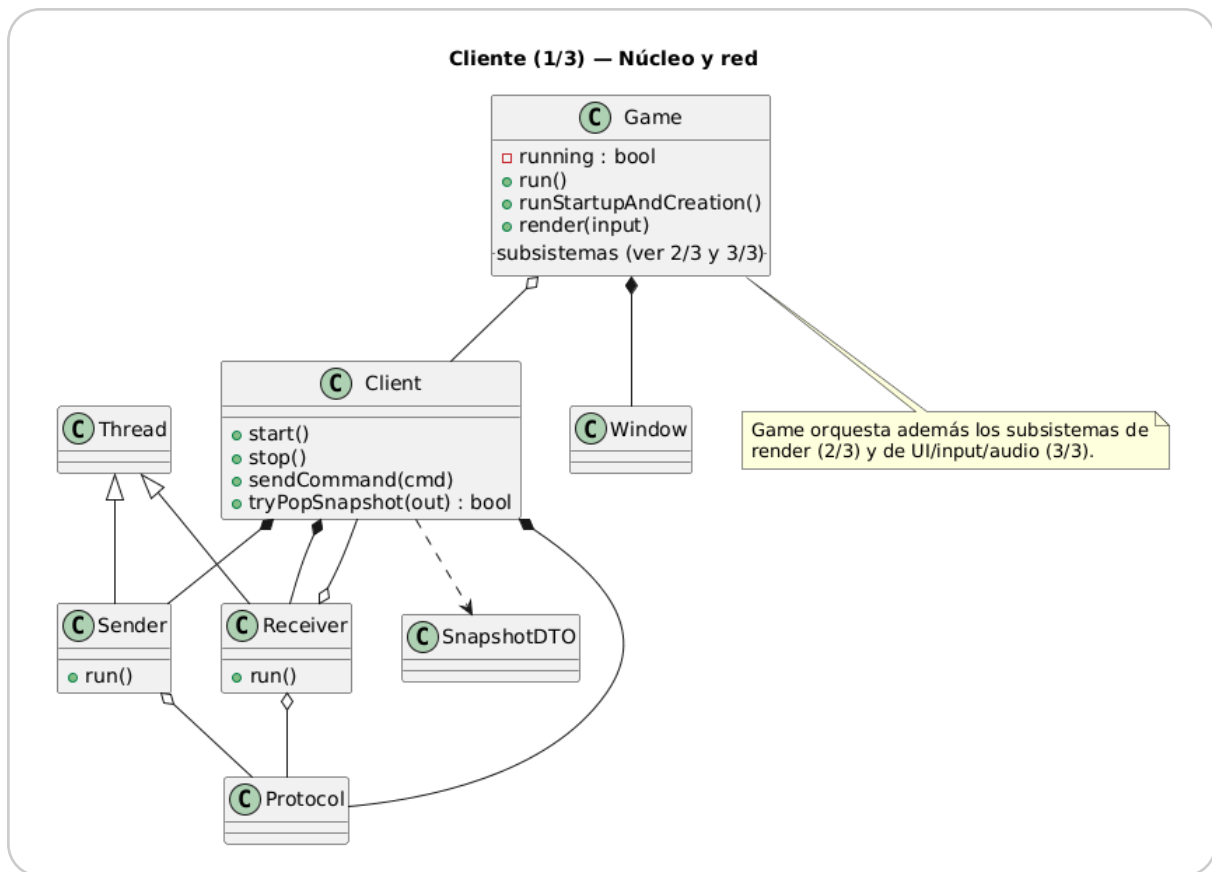


Figura 14: *Cliente (1/3): núcleo y red.* *Game* orchestra la aplicación y se comunica con el servidor a través de *Client*, que corre los hilos *Receiver* y *Sender* sobre el *Protocol* y expone los snapshots al loop de juego.

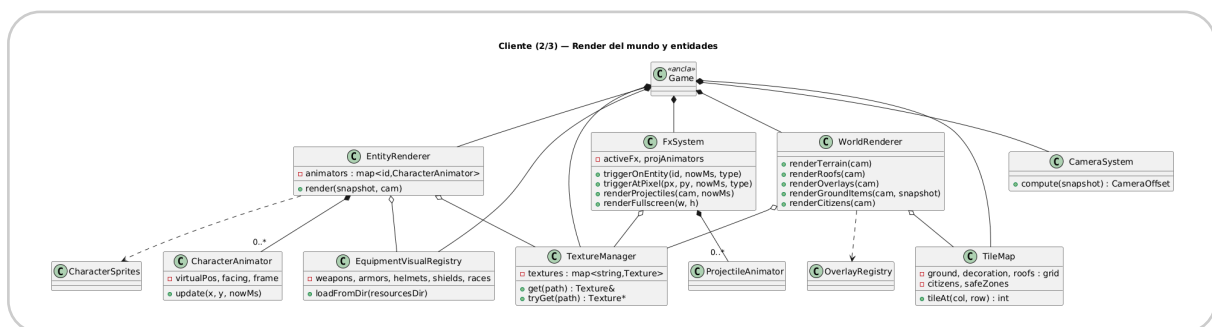


Figura 15: *Cliente (2/3): subsistema de render.* Colgando de *Game*, el *WorldRenderer* dibuja el terreno y los ítems del piso, y el *EntityRenderer* anima a las entidades mediante un *CharacterAnimator* por entidad; ambos se apoyan en *TextureManager*, *TileMap* y *EquipmentVisualRegistry*, con efectos a cargo de *FxSystem*.

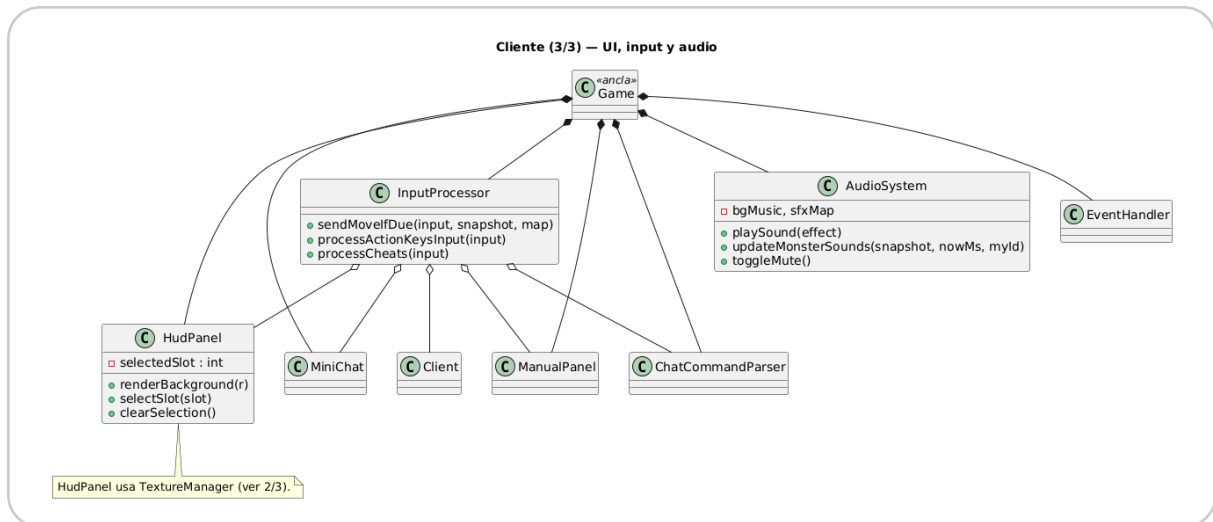


Figura 16: Cliente (3/3): interfaz, entrada y audio. Game compone el HudPanel, el chat (MiniChat), el ManualPanel y el AudioSystem; el InputProcessor traduce las teclas en comandos al Client e interactúa con los paneles y el ChatCommandParser.

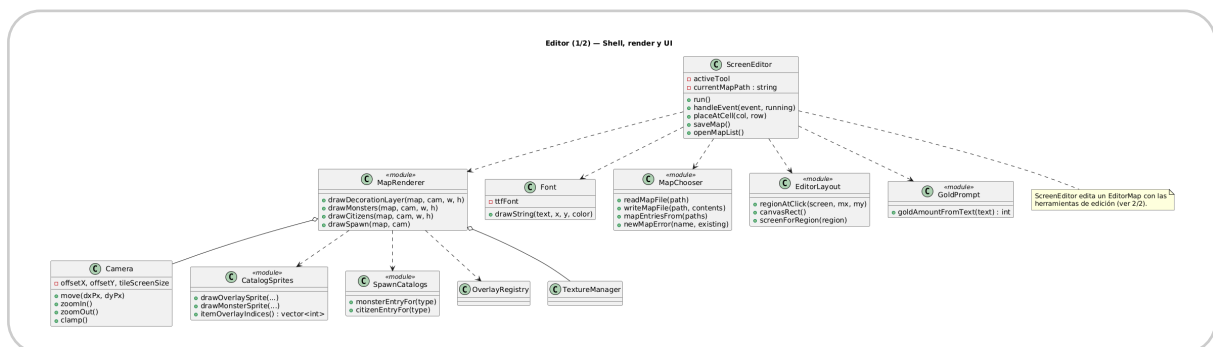


Figura 17: Editor (1/2): shell, render y UI. ScreenEditor maneja el loop y los eventos, dibuja el mapa con MapRenderer (apoyado en Camera, CatalogSprites y SpawnCatalogs) y delega tareas auxiliares en Font, MapChooser, EditorLayout y GoldPrompt.

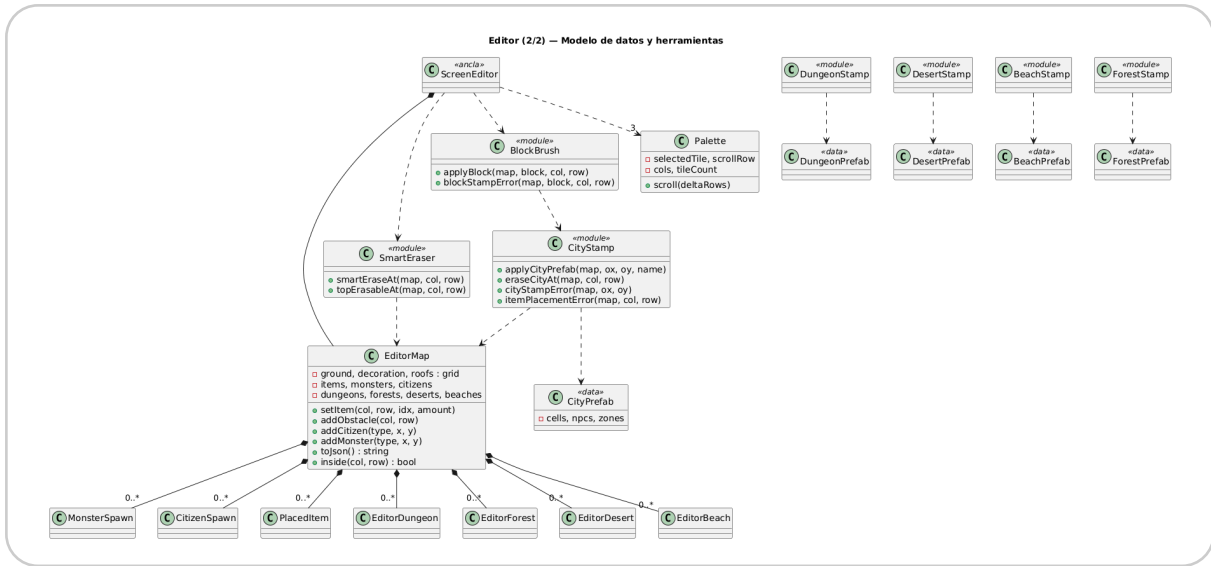


Figura 18: *Editor (2/2): modelo de datos y herramientas.* **ScreenEditor** edita un **EditorMap** (capas de terreno más spawns, ítems y bloques) usando las herramientas **SmartEraser**, **BlockBrush** y **Palette**; los stamps de ciudad, mazmorra, desierto, playa y bosque estampan sus respectivos ***Prefab** sobre el mapa.

4. Tests

Durante el desarrollo del juego, se implementó una suite de tests para el correcto seguimiento de funcionalidades durante todo el proceso de trabajo.

```

[-----] Global test environment tear-down
[=====] 590 tests from 77 test suites ran. (214048 ms total)
[ PASSED ] 590 tests.
Chequeo de Valgrind completado. Reporte guardado en: build/valgrind/reporte_tests.log

```

Figura 19: *Luego de la ejecución de los tests, se observa la siguiente salida, tanto en desarrollo como en el proceso de instalación. En este caso, se observa la ejecución con valgrind*

5. Valgrind

Se observa el reporte de la corrida tests y de una sesión del servidor completa, desde que arranca una partida hasta que termina.

```
vboxuser@Ubuntu26-04:~/taller/ArgentumOnline-Taller$ make valgrind-show
=== REPORTE TESTS ===
==4633== Memcheck, a memory error detector
==4633== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==4633== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==4633== Command: ./argentum_online_tests
==4633== Parent PID: 4632
==4633==
==4633== HEAP SUMMARY:
==4633==   in use at exit: 0 bytes in 0 blocks
==4633==   total heap usage: 312,646 allocs, 312,646 frees, 74,320,949 bytes allocated
==4633==
==4633== All heap blocks were freed -- no leaks are possible
==4633==
==4633== For lists of detected and suppressed errors, rerun with: -s
==4633== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
=== REPORTE SERVER ===
==5385== Memcheck, a memory error detector
==5385== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==5385== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==5385== Command: ./argentum_online_server 8080 --create DefaultWorld --map maps/defaultMap.json
==5385== Parent PID: 5384
==5385==
==5385== HEAP SUMMARY:
==5385==   in use at exit: 0 bytes in 0 blocks
==5385==   total heap usage: 292,242 allocs, 292,242 frees, 164,393,779 bytes allocated
==5385==
==5385== All heap blocks were freed -- no leaks are possible
==5385==
==5385== For lists of detected and suppressed errors, rerun with: -s
==5385== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figura 20: *Reporte de los tests y del server con valgrind.*